CoU-BRACNet Inter University Programming Contest 2023

https://toph.co/c/cou-bracnet-inter-university-2023

Toph

Schedule

The contest will run for **4h15m0s**.

Authors

The authors of this contest are bappy.cse38, kissu_parina, Mubin_BUBT38, Nirjhor, Rafsan.7492, Rajib_119, Reayz77, ridzz007, salman.exe, and ssavi1994.

Rules

This contest is formatted as per the official rules of ICPC Regional Programming Contests.

You can use C# Mono 6.0, C++17 GCC 13.2, C++20 GCC 13.2, Java 1.8, C++11 GCC 7.4, C++14 GCC 8.3, C11 GCC 12.1, C11 GCC 9.2, PyPy 7.1 (2.7), PyPy 7.1 (3.6), Python 2.7, Python 3.11, and Python 3.7 in this contest.

Be fair, be honest. Plagiarism will result in disqualification. Judges' decisions will be final.

Notes

There are 9 challenges in this contest.

Please make sure this booklet contains all of the pages.

If you find any discrepancies between the printed copy and the problem statements in Toph Arena, please rely on the latter.

A. Where is the Ghost

Limits 1s, 256 MB

The ghost exploration team find some **1D** coordinate where the cursed object is located. You will be given N such cursed positions. All the locations are integers. You need to find the possible ghost spawning area using those cursed object positions.

The ghost has a special affection for numbers 1 to 9. An area is said to be a ghost spawning area if it is **divisible** by all the numbers from 1 to 9.

Being a member of the ghost exploration team, you need to find out the number of ways of selecting one or more cursed object locations from those N locations such that the product of those locations returns a ghost spawning area.

Since the answer can be very big, output the answer modulo $10^9 + 7$.

Input

The first line of input is an integer N. The next line will contain N different integer numbers.

Constraints

- $1 \leq N \leq 10^4$
- $1 \leq$ all cursed positions $\leq 10^4$

Output

Print the number of ways of selecting one or more cursed object locations modulo $10^9 + 7$ such that the product of those locations becomes a ghost spawning area.

Examples

Input	Output
7 1 2 3 4 5 6 7	4

Input	Output
6 1 2 3 4 5 6	0

B. A Lazy Statement

Limits 10s, 256 MB

Milk has created an account in the newly launched social network Coffeebook. However, she cannot decide on a password for her account. So she's asked Mocha for help (don't ask me why she trusts Mocha this much).

Mocha has decided to make the process interesting for himself. At each step, he applies the following procedure to come up with a random number.

- He creates a **non-empty** array A of **at most** N integers. Formally, $1 \leq |A| \leq N$.
- The array elements should be positive integers at most M. Formally, $1 \leq A_i \leq M$ for each $1 \leq i \leq |A|$.
- Each element of the array (except the first one) should be divisible by the previous element. Formally, $A_i \mid A_{i+1}$ should hold for each $1 \leq i < |A|$.
- After creating such an array, Mocha raises **the last** element of the array to the power K. This gives him a random number.

Mocha doesn't want to bore himself with the same array over and over again. So at each step, he creates **a new array** (satisfying the above conditions), an array he hasn't created before. He stops only when he runs out of arrays satisfying the above conditions. Then he adds up all the powers he's extracted out of these arrays and adds them up. The result, modulo $10^9 + 7$, is the password he'll suggest to Milk.

However, soon after coming up with this plan, Mocha realizes that it will take him a long time to go through all possible arrays. He can't do it on his own. Can you help him?

Input

The only line of input will contain three space-separated positive integers N – the limit on the size of the array, M – the limit on the array elements, K – the power Mocha raises the last element of each array to.

These integers satisfy $1 \leq N, M \leq 10^9$ and $0 \leq K \leq 100$.

Output

Print a single line containing the password, which is the sum (modulo $10^9 + 7$) of the last elements of all possible arrays satisfying the aforementioned conditions, raised to the power K.

Examples

Input	Output
1 3 0	3
There are only three possible arrays in this case: $[1], [2], [3]$. The power to raise to is $K = 0$. So the password would be $1^0 + 2^0 + 3^0 = 3$.	

Input	Output
2 2 3	26
There are five possible arrays: $[1], [2], [1, 1], [1, 2], [2, 2]$. The power to raise to is $K = 3$. So the password would be $1^3 + 2^3 + 1^3 + 2^3 + 2^3 = 26$.	

Input	Output
314 15 92	168177682
Remember to reduce the result modulo $10^9 + 7$.	

C. Nitu and Her Lucky String

Limits 2s, 512 MB

Recently, Nitu has developed a keen interest in solving string-related problems. She possesses a special 'lucky' string. Just last week, Nitu stumbled upon a fascinating puzzle involving a square matrix, consisting of N rows and N columns. Each cell within this matrix contains only lowercase letters. Her objective is to navigate from the top-left cell $(0, 0)$ to the bottom-right cell $(N-1, N-1)$ while adhering to the following conditions:

- Starting from any cell (i, j), she can only move either to the right $(i, j+1)$ or downward $(i+1, j)$.
- During her journey from cell $(0, 0)$ to cell $(N-1, N-1)$, she must collect all the letters encountered along the way. Subsequently, she will construct a string using these letters, and this string must contain her 'lucky' sequence at least M times as a substring.

While Nitu acknowledges that there might be multiple paths to reach her destination (from $(0, 0)$ to $(N-1, N-1)$), she claims to have solved the problem in a single attempt and is now challenging you. Are you prepared to accept her challenge?

Input

The first line comprises an integer, T $(1 \leq T \leq 100)$, representing the number of test cases.

For each test case:

The first line includes two integers, N $(1 \leq N \leq 100)$ and M $(0 \leq M \leq 10)$.

The subsequent N lines contain strings of length N, forming a square matrix.

Following the matrix description, there are additional lines containing her "lucky" string contains only lowercase letters and $1 \leq |\text{lucky}| \leq 100$ where $|\text{lucky}|$ means the length of “lucky“ string.

Output

For each test case, please output a single line in the following format:
`Case #`X`:` Y.

Here X represents the test case number, and Y represents the number of paths to reach Nitu's destination (from cell $(0, 0)$ to cell $(N-1, N-1)$) modulo $10^9 + 7$.

Example

Input	Output
2 3 2 abc bbc abb bbb 4 3 abcd bbcd abbc xyzp bbb	Case #1: 2 Case #2: 0
In the first input, Nitu can go (0,0) -> (0,1) -> (1,1) -> (2,1) -> (2,2) From this path she can make a string abbbb. This string contains her lucky string bbb 2 times as a substring.	

Notes

A substring is a contiguous sequence of characters within a larger string.

D. Profitable Home

Limits 4s, 64 MB

Milk and Mocha want to move to a new house in Coffeeland. They don't like ugly corners sticking into their house, so they want the shape of the house to be a convex polygon with a positive area. Mocha has already shortlisted N distinct points where the pillars of the house can be placed at. They just need to choose a few of them and build walls around them.

The transactions in Coffeeland happen with coffee beans. Mocha has already contacted a construction company to build the walls for them. The company doesn't charge for the pillars. To build a wall between two pillars, the company charges the number of coffee beans equal to the **squared Euclidean distance** between the pillars. The cost to build a house would therefore be the sum of the costs to build each such wall.

Milk and Mocha like open spaces. However, financially they're very aware. They'd like to get the best out of their investment. That's why, they'd choose a house that maximizes the **area per every coffee bean** they'd be investing. Can you find the maximum possible value for them? You need to do it for T construction sites that they have in mind.

Input

The first line contains T, the number of independent construction sites Milk and Mocha have in mind. Then the descriptions of these T cases follow.

The first line of each case contains the integer N, the number of points where pillars can be placed. This number satisfies $3 \leq N \leq 200$.

The next N lines contain two space-separated integers x and y each, denoting the coordinates of the chosen points. These numbers satisfy $-1000 \leq x, y \leq 1000$. **No three** of these N points are collinear.

The total number of points over all these cases does not exceed 200.

Output

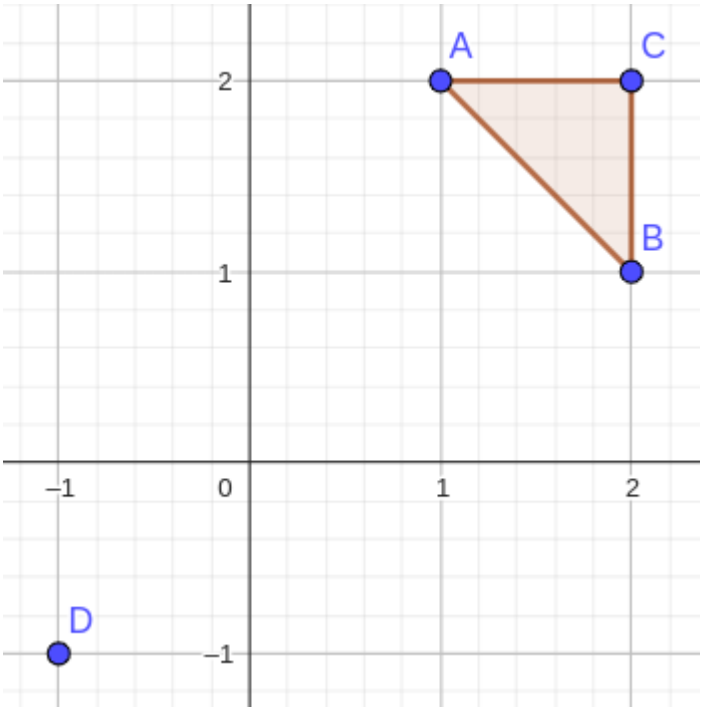
For each case, print a single real number — the maximum possible value of **area per coffee bean** they can get if they choose their house wisely.

Your answer would be considered correct if the absolute difference to the actual answer is within 10^{-6}.

Example

Input	Output
2 4 1 2 2 1 2 2 -1 -1 4 0 0 5 0 0 5 5 5	0.125 0.25

The points in the first sample case represent the points A, B, C, D respectively in the following image.



One can check that it's the most profitable to build a house around the triangle ABC, giving us the value of $\frac{\triangle ABC}{|AB|^2 + |BC|^2 + |CA|^2} = \frac{0.5}{(1^2 + 1^2) + (0^2 + 1^2) + (1^2 + 0^2)} = 0.125$ for **area per coffee bean**. Here they get an area of 0.5 for 4 coffee beans.

In particular, choosing to build a house around all the points (i.e. choosing $ADBC$) brings them a bigger area, but also costs a lot more, leading to a smaller value of area per coffee bean.

Input	Output
The points in the second sample are the vertices of a square, and building the house around all the points is one optimal solution.	

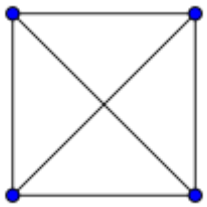
E. Queries, Queries Everywhere

Limits 1s, 512 MB

You are given an array a of length n. You are also given q queries on the array and you have to process them. Queries are of two types:

- $1\ \ i\ \ v$: Set $a[i] = v$.
- $2\ \ L\ \ R$: Construct a **complete** graph on exactly $R - L + 1$ nodes $L, L + 1, \ldots, R$. The graph is complete, meaning **every node is connected to every other node by an edge**. The weight of the edge between two nodes is the maximum of their values. Formally, the edge between nodes u, v is $\max(a[u], a[v])$, where $L \leq u < v \leq R$. You have to print the **cost** of the minimum spanning tree (MST) of this graph.

Here is an example of an **unweighted** complete graph with 4 nodes (note that in our case the **edges will have weights**).



A spanning tree of a graph is a subset of edges taken from the graph such that they form a tree and every vertex on the graph is reachable from every other vertex via the chosen set of edges. The *cost* of a spanning tree is just the sum of edge weights in it. A minimum spanning tree of a graph is a spanning tree with the minimum cost.

Input

The first line contains the integer T $(1 \leq T \leq 100000)$, the number of test cases. For each test case:

The first line contains the integers n $(1 \leq n \leq 200000)$ and q $(1 \leq q \leq 100000)$.

The second line contains n integers describing the contents of the array a $(1 \leq a[i] \leq 2 \times 10^9)$.

The next q lines are the queries you have to process in one of the following forms:

- $1\ \ i\ \ v$: These integers satisfy $1 \leq i \leq n$ and $1 \leq v \leq 2 \times 10^9$. In this case, you have to set $a[i] = v$.

- 2 L R: These integers satisfy $1 \leq L \leq R \leq N$. In this case, you have to print the cost of the MST in the graph described above.

The sum of n over all test cases does not exceed 2×10^5. The sum of q over all test cases does not exceed 10^5.

Output

For each query of the second type, output a single number in one line - the cost of the MST of the graph constructed.

Don't print an extra newline after each test case. Every case has at least one query of the second type.

Example

Input	Output
1 5 6 1 2 3 4 5 1 1 9 1 2 8 1 3 3 1 4 3 1 5 11 2 2 5	22
There can be multiple ways to construct an MST.	

F. Safwan's Book Bonanza

Limits 2s, 512 MB

A grand book fair is taking place in your city, showcasing a dazzling array of books and publications. Safwan is a passionate book lover going to this book fair with a goal to buy as many books as possible.

There are n unique books available at the fair, each with its own price and publication id. Also, there are m publications participating in the fair, each offering a special **discount** on the total amount of the books you bought from them. Suppose you bought total 2 books with prices $20, 30$ from publication 1 and publication 1 is offering a discount amount of 60. Then you can buy these 2 books for free. If the books price are $30, 40$, then you can buy these books spending $30 + 40 - 60 = 10$ amount.

Safwan will be asked q queries. In each query, he will be given an **amount.** To maximize his reading pleasure, he will always try to purchase the maximum number of books with the given amount. Would you like to help him?

N. B. You can't buy the same book multiple times.

Input

The first line of the input contains three integers $1 \leq n \leq 10^6, 1 \leq m \leq 10^6, 1 \leq q \leq 10^5$. Note that, m is always less than or equal to n and there is at least one book of every publication.

Each of the next n lines consists of two integers $\texttt{price}$ $(1 \leq \texttt{price} \leq 10^{12})$ and $\texttt{publicationId}$ $(1 \leq \texttt{publicationId} \leq m)$ of i-th book.

Next line contains an array $\texttt{discounts}$ of m integers $\texttt{discounts}[1], \texttt{discounts}[2], \ldots, \texttt{discounts}[m]$ where $0 \leq \texttt{discounts}[i] \leq 10^{18}$ means i-th publisher will offer this discount if you buy one or more books from their collection.

Each of the next q lines contains an integer which denotes the amount $(0 \leq \texttt{amount} \leq 10^{18})$ of that query.

Output

For each query, print maximum number of books Safwan can buy with the given amount of that query. See the sample input output for more clarification.

Example

Input	Output
5 2 3 5 1 5 1 6 1 7 2 12 2 6 0 23 35 50	4 5 5

G. The Clash of Tactical Titans: Klopp vs. Ten Hag

Limits 1s, 512 MB



In the intense world of football rivalries, where victory and defeat carry immense weight, two iconic tacticians have emerged as the focal points of a legendary showdown: Jürgen "Prime Dominator" Klopp, the unyielding coach of Liverpool FC, and Erik "Prime Struggler" ten Hag, the enigmatic leader of Manchester United. Their rivalry centers around a peculiar belief - the influence of prime numbers on the outcome of matches.

Part 1: Klopp's Legendary Belief

Coach Klopp, revered for his charismatic leadership and dynamic playing style, has an unshakable belief in the concept of Prime Victory Blitz Days. According to Klopp, these are extraordinary dates when both the day and the month are prime numbers, destined to bring triumph to Liverpool FC. Klopp's relentless pursuit is to find these dates, as he believes they are the key to Liverpool's unparalleled success.

Part 2: Ten Hag's Troubles

On the opposing side, Coach Ten Hag, celebrated for his tactical prowess, has been burdened by an unusual belief in what he dubs "Prime Defeat Days." Ten Hag is convinced that on dates where both the day and the month are prime numbers, Manchester United is destined to face defeat. He is on a mission to avoid these days at all costs, for he believes they spell doom for his team.

Part 3: The Fateful Encounter

As fate would have it, both coaches have circled a significant date on their calendars: 05/03/2023. On this day, Liverpool FC is set to square off against Manchester United in a high-stakes, international club competition. For Klopp, it's a Prime Victory Blitz Day, while for Ten Hag, it's a dreaded Prime Defeat Day.

Part 4: The Clash on 05/03/2023

The football world is abuzz with anticipation as Klopp and Ten Hag face off on 05/03/2023. It's a match for the ages, filled with tension, drama, and extraordinary moments. As the clock ticks, both teams give their all, and the outcome remains uncertain until the final whistle.

Part 5: The Outcome



The match ends in a spectacular 7-0, leaving fans in awe of the display of skill and tactics. Klopp celebrates what he calls another Prime Victory Blitz Day, while Ten Hag breathes a sigh of frustration!

Part 6: The Continuing Rivalry

The rivalry between Klopp and Ten Hag continues, each coach seeking to prove their theory about prime numbers. Klopp hunts for Prime Victory Blitz Days to cement Liverpool's dominance, while Ten Hag strategizes to avoid Prime Defeat Days and lead Manchester United to victory.

Write a program that assists Klopp and Ten Hag in their quest to identify Prime Victory Blitz Days and Prime Defeat Days for their respective teams. Your task is to calculate the total count of these special dates within a given range of years, as both coaches aim to shape the destiny of their teams.

Input

The first line comprises an integer, T $(1 \leq T \leq 20100)$, representing the number of test cases.

For each test case there are two positive integers: the starting year s, and the ending year e, where$10^2 \leq s \leq e \leq 10^{12}$.

Output

An integer representing the count of Prime Victory Blitz Days for Liverpool FC within the given range.

Example

Input	Output
2 1900 1902 2022 2023	156 104

H. Nitu and Pseudo-code

Limits 1s, 512 MB

Last night, Nitu came across the following pseudo-code. She understood that this method would take two integer values and a list of integers as input. She decided to implement this method in her preferred programming language and tried calling it with various sets of values. However, she encountered performance issues with her implementation for certain input values, as it did not execute within the expected timeframe. Can you assist her in implementing this pseudo-code?

```
FUNCTION CountLuckyNumbers(l, r, forbidden_list)
    # Initialize a counter for lucky numbers
    count_lucky_numbers = 0

    # Loop through the range from l to r (inclusive)
    FOR i FROM l TO r
        # Convert the current number to a string
        current_number = CONVERT_TO_STRING(i)

        # Initialize a flag to determine if the number is allowed
        is_allowed = TRUE

        # Check if the current number contains any of the forbidden numbers
        FOR EACH forbidden IN forbidden_list
            # Convert the forbidden number to a string for comparison
            forbidden_string = CONVERT_TO_STRING(forbidden)

            # If the current number contains the forbidden number as a substring,
            # Set the flag to false and exit the loop
            IF current_number CONTAINS forbidden_string
                is_allowed = FALSE
                BREAK
            END IF
        END FOR

        # If the current number is allowed, increment the lucky number count
        IF is_allowed
            INCREMENT count_lucky_numbers BY 1
        END IF
    END FOR

    # Return the count of lucky numbers
    RETURN count_lucky_numbers
END FUNCTION
```

Input

The input begins with an integer, T $(1 \leq T \leq 100)$ indicating the number of test cases.

For each test case:

1. The first line contains an integer, N $(1 \leq N \leq 100)$ which represents the size of the forbidden list.
2. The subsequent N lines contain integers, each representing an element in the forbidden list. Each of these elements falls within the range 1 to 10^{18}.

3. Finally, there is a line containing two integers, l and r where both l and r are within the range of 1 to 10^{18}.

Output

For each test case, please provide a single line of output in the following format: `Case #`X`: `Y.

Here, X represents the test case number, and Y represents the result returned by the pseudocode.

Example

Input	Output
1 2 3 6 1 10	Case #1: 8

I. Alice and Bob's Laziness

Limits 2s, 512 MB

Father works for rail stations and his job is to supply and monitor equipment at the stations. The rail stations are connected with bidirectional railways, and each station can be reached from all other stations without forming any cycles. The equipment is measured by weight.

Father has to supply the equipment to the stations and also monitor the equipment weights frequently. He visits the stations in a manner where he checks all the station equipment from a starting station to a destination station. Sometimes, the equipment becomes unusable or problems occur, and the station master reports these issues. Father then replaces the equipment with new ones (not necessarily of the same weight) quickly.

Nowadays, Father is suffering from sickness, and he wants his two lazy sons, Alice and Bob, to take over his work. Despite their reluctance, Father divides the tasks between them. He assigns Alice to supply the equipment and Bob to monitor the equipment.

Alice and Bob are lazy, and they do not perform their tasks properly. When Father asks Alice to supply equipment to a station, **Alice changes the weight of the equipment by its number of divisors**.

When Father asks Bob to monitor equipment from station U to station V, **Bob only monitors those equipment weights that he hasn't monitored earlier during this specific visit from U to V**. He skips some stations' equipment and only checks stations further along the path from U to V.

Father does not care about the equipment weight changed by Alice because it is easy to figure out. However, he wants to know how many stations' equipment Bob skipped each time Father asks Bob to monitor.

There are N stations numbered from 1 to N, connected by $N - 1$ roads. One can reach every station from every other using the roads. Initially, Father gives Alice an array of weights S, and Alice is supposed to supply the equipment with weight S_i to the i-th station for $1 \leq i \leq N$.

There will also be Q tasks. A task will be for Alice if it starts with `A` or it will be for Bob if it starts with `B`.

For Alice's task, a station with the number U and the equipment weight W will be given. Alice is supposed to supply the equipment with weight W to the station U.

For Bob's task, a starting station U and a destination station V will be given. Bob has to monitor the equipments along the simple path from U to V.

Since Bob always skips the weight he did monitor earlier during that specific visit from U to V, Father asks you to **find the number of stations whose equipment Bob skipped**.

Input

The first line contains two integers N and Q, representing the number of stations and the number of tasks, respectively.

The second line contains an array S of N integers, where S_i indicates the initial equipment weight of the i-th station, which is supposed to be supplied by Alice initially.

The next $N-1$ lines describe the connections between stations. Each line contains two integers U and V, indicating a connection between stations U and V.

Finally, there will be Q lines describing the tasks. Each task is either of the form `A` U W or `B` U V.

Constraints

- $1 \leq N, Q, U, V \leq 2 \times 10^5$
- $1 \leq S_i, W \leq 10^6$

Output

For each task of type `B` U V, you have to print the number of stations that Bob skipped while visiting U to V.

Example

Input	Output
8 7 10 12 16 20 15 26 25 7 1 2 3 1 4 8 1 4 2 5 6 7 2 6 B 5 7 B 6 3 B 7 8 A 4 24	1 1 2 1 2

Input	Output
B 7 8 A 2 21 B 5 7	
After the changes of Alice, the initial S will be $[4, 6, 5, 6, 4, 4, 3, 2]$.	

Notes